

**UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA
FILHO”**

CÂMPUS DE SOROCABA

CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO

RELATÓRIO TÉCNICO FINAL

Projeto de Estrutura de Dados — ED2026

**ECOSSISTEMA DE MONITORAMENTO INDUSTRIAL DE
SENSORES COM ANÁLISE COMPARATIVA DE ESTRUTURAS DE
DADOS AVANÇADAS**

Disciplina: Estrutura de Dados

Discente: Enrico Rodrigues Pereira

Instituição: UNESP — Câmpus de Sorocaba

Sorocaba, 29 de junho de 2026

Sumário

1	Introdução e Objetivos do Sistema	1
2	Modelagem dos Dados: Entradas, Saídas e Menu Interativo	1
2.1	Estrutura de Entrada: o Tipo Abstrato <code>LeituraSensor</code>	1
2.2	Mecanismo de Saída: Pipeline de <i>Benchmark</i>	2
2.3	Menu Interativo de Consultas	2
3	Metodologia e Tratamento Estatístico de Dados	3
3.1	Geração Estocástica: Transformação de Box-Muller	3
3.2	Tratamento de Anomalias: Cerca Distante de Tukey por IQR	3
3.3	Nota Crítica: Resiliência Algorítmica	4
4	Arquitetura do Software e Detalhamento das Estruturas	4
4.1	Estruturas Tradicionais da Ementa	4
4.1.1	Lista Encadeada Linear (<code>ListaSensores</code>)	4
4.1.2	Tabela Hash com Encadeamento Externo (<code>TabelaHashSensores</code>)	5
4.1.3	Árvore AVL (<code>ArvoreAVLSensores</code>)	5
4.2	Estruturas Avançadas Fora da Ementa	5
4.2.1	Skip List Probabilística (<code>SkipListSensores</code>)	5
4.2.2	Cuckoo Hashing (<code>CuckooHash</code>) — Estrutura Otimizada	6
4.2.3	Segment Tree (<code>SegmentTreeSensores</code>)	6
4.3	Gerenciador de Alertas: Max Heap (<code>HeapSensores</code>)	6
5	Otimização: Hash Tradicional vs. Cuckoo Hashing	6
5.1	Busca $O(1)$ no Pior Caso Absoluto	6
5.2	Inserção por Expulsão (Efeito <i>Ping-Pong</i>)	7
5.3	Trade-off Empírico: Inserção vs. Busca	7
6	Discrepância entre Teoria Ideal e Realidade Prática	8
6.1	Custo das Constantes Multiplicativas	8
6.2	Hierarquia de Memória Cache	8
7	Mapeamento de Restrições em Condições Críticas	9
7.1	Categoria 1 — Memória: Restrição R5	9
7.2	Categoria 2 — Processamento: Restrição R10	9
7.3	Categoria 3 — Latência: Restrição R12	9
7.4	Categoria 4 — Dados: Restrições R16 e R18	10
7.5	Categoria 5 — Algorítmica/Estrutural: Restrição R21	10
8	Delimitação Metodológica: Grupo A vs. Grupo B	10
8.1	Por que Grupo A e Grupo B são Incomparáveis	10
9	Resultados Experimentais e Benchmark Assintótico	12

9.1	Tabela 1: Desempenho Operacional para $N = 19.983$	12
9.2	Tabela 2: Escalabilidade e Razão de Crescimento Empírico	13
9.3	Tabela 3: Consumo de Memória RAM	14
9.4	Figura 1: Comparativo de Tempos por Operação — $N = 2.000$	14
9.5	Figura 2: Comparativo de Tempos por Operação — $N = 10.000$	15
9.6	Figura 3: Comparativo de Tempos por Operação — $N = 19.983$	16
9.7	Figura 4: Curva de Escalabilidade do Tempo de Busca — Grupo A	17
9.8	Análise de Complexidade Assintótica Real	18
10	Conclusão	18
11	Documentação do Uso de Inteligência Artificial Generativa	19

1 INTRODUÇÃO E OBJETIVOS DO SISTEMA

O avanço acelerado da Indústria 4.0 impõe aos sistemas computacionais modernos requisitos cada vez mais rigorosos de eficiência no processamento, armazenamento e recuperação de grandes volumes de dados heterogêneos gerados em tempo real. Nesse contexto, a telemetria industrial, definida como a coleta e o monitoramento remoto de grandezas físicas por meio de sensores distribuídos, constitui um dos domínios de aplicação mais exigentes para o projeto de algoritmos e estruturas de dados, uma vez que a latência de acesso à informação, a resiliência a falhas de hardware e a escalabilidade sob crescimento contínuo do fluxo de dados são requisitos funcionais críticos, e não meramente desejáveis.

O presente trabalho documenta o desenvolvimento de um ecossistema de software completo em linguagem C, concebido para simular um sistema de monitoramento industrial capaz de processar e indexar até 25.000 amostras sintéticas de leituras de sensores industriais. O sistema foi projetado com a premissa de que ambientes industriais reais são intrinsecamente ruidosos e sujeitos a falhas: sensores defeituosos geram valores anômalos, redes de transmissão perdem pacotes e o hardware de armazenamento opera sob restrições de memória RAM. A modelagem dessas condições adversas é, portanto, central à arquitetura da solução proposta.

O objetivo geral deste projeto é demonstrar, de forma empírica e rigorosa, como a escolha da estrutura de dados impacta diretamente o desempenho de um sistema computacional real, indo além da análise assintótica puramente teórica. Para tanto, o ecossistema implementa, integra e compara sete estruturas de dados distintas, sendo das quais três pertencentes à ementa regulamentar da disciplina e três extracurriculares, submetendo-as a um protocolo de benchmark padronizado que mede tempos de inserção, busca e remoção em múltiplas escalas de entrada.

Os objetivos específicos compreendem: (i) a implementação correta e completa das operações fundamentais de inserção, busca e remoção sobre todas as estruturas integradas; (ii) o desenvolvimento de três operações analíticas adicionais sobre o dataset; (iii) a geração de dados sintéticos realistas suficientes para evidenciar diferenças de desempenho; (iv) a aplicação de cinco categorias de restrições operacionais que simulam condições adversas reais; e (v) a apresentação de um benchmark comparativo metodologicamente válido, distinguindo grupos de estruturas cujas finalidades computacionais são efetivamente comparáveis.

2 MODELAGEM DOS DADOS: ENTRADAS, SAÍDAS E MENU INTERATIVO

2.1 Estrutura de Entrada: o Tipo Abstrato `LeituraSensor`

A unidade fundamental de informação manipulada pelo sistema é representada pelo tipo abstrato de dado `LeituraSensor`, definido como uma estrutura C com três campos semanticamente distintos. O campo `timestamp` (`long long`) atua como chave primária de ordenação e identificação unívoca, armazenando um instante temporal no formato UNIX. O campo `sensor_id` (`int`) identifica o sensor físico de origem. O campo `valor` (`double`) armazena a grandeza física

medida — temperatura em graus Celsius, neste projeto.

```

1 typedef struct {
2     long long timestamp;    /* Chave primaria: instante UNIX da leitura
                               */
3     int      sensor_id;     /* Identificador do sensor fisico (1 a 5)
                               */
4     double   valor;         /* Grandeza medida: temperatura em graus C
                               */
5 } LeituraSensor;

```

Listing 1: Definição do tipo abstrato `LeituraSensor`

O `timestamp` foi eleito como chave primária por três razões técnicas: (i) unicidade natural; (ii) ordenabilidade total, viabilizando árvores AVL e Skip List; e (iii) compatibilidade direta com operações de *hash* aritmético sem conversões de tipo. Todas as estruturas implementadas utilizam o `timestamp` como chave de indexação, garantindo uma interface uniforme para fins de *benchmark*.

2.2 Mecanismo de Saída: Pipeline de *Benchmark*

O sistema consolida resultados por meio de um *pipeline* de *logging* implementado em `rodarBenchmarkGlobal`, registrando tempos de execução em milissegundos via `obter_tempo_ms`. Em sistemas POSIX, utiliza-se `gettimeofday` (resolução de microssegundo); em Windows, `QueryPerformanceCounter` (resolução submicrossegundo). As saídas incluem tabelas comparativas, gráficos de barras ASCII normalizados por `imprimirGraficoBarras` e análise assintótica empírica calculada por `analisarComplexidadeReal`, que compara a razão de crescimento real dos tempos medidos com as razões teóricas para $O(1)$, $O(\log n)$ e $O(n)$.

2.3 Menu Interativo de Consultas

Em conformidade com a exigência do enunciado de “consultas interativas”, o software implementa `rodarMenuInterativo`, acionada antes da execução automática dos *benchmarks*. O menu instancia uma `TabelaHashSensores` local, populada com todos os elementos do *dataset*, provendo buscas $O(1)$ em tempo de interação.

```

1 do {
2     printf("Opcao: ");
3     if (scanf("%d", &opcao) != 1) { while (getchar() != '\n'); continue;
4     }
5     switch (opcao) {
6         case 1: /* Inserir nova leitura (timestamp via time(NULL)) */
7         case 2: /* Buscar por timestamp - O(1) via Hash */
8         case 3: /* Remover por timestamp */
9         case 4: /* calcularEstatisticasGerais() [OP 1] */

```

```

9      case 5: /* filtrarEOrdenarAlertasCriticos() [OP 3] */
10     case 6: /* preverTendenciaSensor() [OP 2] */
11     case 0: /* Sair e iniciar simulacao automatica */
12     }
13 } while (opcao != 0);

```

Listing 2: Estrutura lógica do laço *switch-case* do menu

As três operações analíticas obrigatórias do enunciado estão disponíveis como opções do menu (case 4, 5, 6) e também são executadas automaticamente na fase de simulação. A opção 0 encerra o menu e transfere controle ao *benchmark*.

3 METODOLOGIA E TRATAMENTO ESTATÍSTICO DE DADOS

3.1 Geração Estocástica: Transformação de Box-Muller

O sistema gera 25.000 amostras via `gerarDatasetRestrito`, modelando cinco sensores industriais de temperatura em regime contínuo com valores ao redor de 30–40 °C e ruído gaussiano realista produzido por `aplicarRuido`.

O ruído é produzido pela **Transformação de Box-Muller** [6], que converte dois números uniformes $U_1, U_2 \sim \mathcal{U}(0, 1]$ em duas variáveis $\mathcal{N}(0, 1)$:

$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_1 = \sqrt{-2 \ln U_1} \sin(2\pi U_2) \quad (1)$$

A implementação em C emprega variáveis estáticas `generate` e `z1` para aproveitar ambas as amostras gaussianas antes de novas chamadas ao gerador uniforme, reduzindo pela metade o custo computacional médio. Com desvio padrão de 1,5 °C, produz-se uma distribuição de temperaturas aproximadamente em [25, 44] °C — intervalo confirmado experimentalmente: Min = 25,71 °C, Max = 44,11 °C.

3.2 Tratamento de Anomalias: Cerca Distante de Tukey por IQR

Na função `calcularLimiarAnomalia`, o sistema calcula dinamicamente um limiar robusto baseado no Intervalo Interquartil $IQR = Q_3 - Q_1$) e na Cerca Distante de Tukey [7]:

$$limiar = Q_3 + 3 \times IQR \quad (2)$$

Para este *dataset*: $Q_1 \approx 31$, $Q_3 \approx 37$, $IQR \approx 6$, resultando em $limiar = 37 + 18 = 55,00$ °C — valor confirmado na saída do terminal. A robustez desse critério ante as 2.082 anomalias em 9.999,99 °C é sua maior vantagem sobre o z-score: enquanto a média e o desvio padrão são fortemente perturbados por valores extremos, Q_1 , Q_3 e o IQR dependem apenas da ordenação relativa dos dados, sendo resistentes a até 25% de contaminação [7].

```

1 double calcularLimiarAnomalia(VetorLeituras* dataset) {

```

```

2   int N = (dataset->tamanho < 500) ? dataset->tamanho : 500;
3   double* amostra = (double*)malloc(N * sizeof(double));
4   if (!amostra) return 1000.0;           /* fallback seguro */
5   for (int i = 0; i < N; i++) {
6       int idx = (int)((long long)i * dataset->tamanho / N);
7       amostra[i] = dataset->dados[idx].valor; /* amostragem
          sistematica */
8   }
9   qsort(amostra, N, sizeof(double), compararDoubles);
10  double q1 = amostra[N/4], q3 = amostra[3*N/4], iqr = q3 - q1;
11  free(amostra);
12  return q3 + 3.0 * iqr;                 /* Equacao (2) */
13 }

```

Listing 3: Cerca Distante de Tukey em calcularLimiarAnomalia

3.3 Nota Crítica: Resiliência Algorítmica

A coexistência do limiar dinâmico ($\approx 55^\circ\text{C}$) com verificações complementares de valores inferiores a 1.000,0 em trechos auxiliares constitui uma decisão consciente de arquitetura de resiliência e não uma inconsistência. A premissa de projeto é que qualquer leitura que atinja ordens de magnitude extremas deve ser isolada independentemente do caminho de código que a detecta. Essa abordagem em camadas, denominada *defense in depth* na engenharia de sistema, é prática recomendada para software de missão crítica: a Cerca de Tukey é o critério primário estatisticamente fundamentado; as verificações complementares atuam como camadas de defesa redundantes.

4 ARQUITETURA DO SOFTWARE E DETALHAMENTO DAS ESTRUTURAS

4.1 Estruturas Tradicionais da Ementa

4.1.1 Lista Encadeada Linear (ListaSensores)

Implementa uma lista simplesmente encadeada com inserção $O(1)$ no topo. Cada nó (NoListaSimples) armazena o dado e um ponteiro prox para o sucessor. A busca (Lista_buscar) é $O(n)$: exige caminhamento sequencial por todos os nós a partir do início. Essa assimetria (inserção trivial vs. busca linear) é explorada pela Restrição R21 (Seção 7).

```

1 void Lista_inserir(ListaSensores* lista, LeituraSensor leitura) {
2     NoListaSimples* novo = (NoListaSimples*)malloc(sizeof(NoListaSimples)
3     );
4     novo->dado = leitura;
5     novo->prox = lista->inicio; /* redireciona cabecalho: O(1) */
6     lista->inicio = novo;
7     lista->tamanho++;
8 }

```

```
7 }
```

Listing 4: Inserção $O(1)$ no topo da Lista Encadeada

4.1.2 Tabela Hash com Encadeamento Externo (*TabelaHashSensores*)

Implementa um mapa de dispersão com resolução de colisões por listas encadeadas. A função `Hash_calcular` mapeia o timestamp de 64 bits para um índice via módulo sem sinal: $h(k) = ((\text{unsigned long long } k) \bmod m)$, onde m é a capacidade configurável. O contador `totalColisoes` registra cada colisão, fornecendo a métrica empírica de qualidade da função `hash`. Para $N = 19.983$ e $m = 39.966$, foram registradas **4.309 colisões** (fator de carga $\alpha \approx 0,50$).

4.1.3 Árvore AVL (*ArvoreAVLSensores*)

Árvore Binária de Busca com balanceamento automático, garantindo altura máxima $O(\log n)$ por meio de rotações. O fator de balanceamento de cada nó NoAVL é $FB(v) = h(v.\text{esq}) - h(v.\text{dir})$; rotações são aplicadas quando $|FB| > 1$.

```
1 NoAVL* AVL_rotacaoDireita(NoAVL* y) {
2     NoAVL* x = y->esquerda;
3     NoAVL* T2 = x->direita;
4     x->direita = y;      /* y desce como filho direito de x */
5     y->esquerda = T2;    /* T2 realocado como filho esq de y */
6     y->altura = max_int(AVL_altura(y->esquerda),
7                         AVL_altura(y->direita)) + 1;
8     x->altura = max_int(AVL_altura(x->esquerda),
9                         AVL_altura(x->direita)) + 1;
10    return x;           /* nova raiz da subarv. rebalanceada */
11 }
```

Listing 5: Rotação simples à direita (caso Esq-Esq) na AVL

4.2 Estruturas Avançadas Fora da Ementa

4.2.1 Skip List Probabilística (*SkipListSensores*)

Estende a lista encadeada ordenada com múltiplas camadas de ponteiros “atalho”, obtendo $O(\log n)$ esperado sem rebalanceamento determinístico [3]. A função `sortearNivel` sorteia geometricamente a altura de cada nó com $P(\text{nível} \geq k) = 0,5^{k-1}$, garantindo em expectativa $n/2^\ell$ nós no nível ℓ . A busca percorre os níveis superiores (atalhos longos) antes de descer aos inferiores (varredura local).

4.2.2 Cuckoo Hashing (CuckooHash) — Estrutura Otimizada

Detalhada integralmente na Seção 5. Garante busca $O(1)$ no pior caso absoluto por meio de duas posições candidatas determinísticas.

4.2.3 Segment Tree (SegmentTreeSensores)

Árvore de segmentos sobre o vetor de valores do *dataset*, otimizada para *Range Maximum Queries* (RMQ) em $O(\log n)$. Armazenada implicitamente em vetor de tamanho $4n$, com filhos do nó i nas posições $2i + 1$ e $2i + 2$.

```

1 double SegmentTree_consultarInterno(SegmentTreeSensores* st,
2     int no, int inicio, int fim, int l, int r) {
3     if (l > fim || r < inicio) return -1.0; /* sem sobreposicao */
4     if (l <= inicio && r >= fim) return st->arvore[no]; /* cob. total */
5     int meio = (inicio + fim) / 2;
6     return max_double(
7         SegmentTree_consultarInterno(st, 2*no+1, inicio, meio, l, r),
8         SegmentTree_consultarInterno(st, 2*no+2, meio+1, fim, l, r));
9 }

```

Listing 6: Consulta de intervalo (RMQ) na Segment Tree

A função `SegmentTree_remove` realiza remoção *lógica*: marca a posição como $-1,0$ e propaga o máximo ascendente. Efeito confirmado experimentalmente: máximo antes da limpeza = $9.999,99$ °C; após remoção de 2.082 anomalias (≥ 55 °C), máximo = $44,11$ °C.

4.3 Gerenciador de Alertas: Max Heap (HeapSensores)

Heap binário máximo sobre vetor contíguo. A organização implícita elimina ponteiros: pai de $i = \lfloor (i - 1)/2 \rfloor$; filhos = $2i + 1$ e $2i + 2$. Inserção via `Heap_subir` (*sift up*) e extração via `Heap_descer` (*sift down*), ambas $O(\log n)$. O vetor contíguo potencializa a localidade de *cache* em extrações sequenciais.

5 OTIMIZAÇÃO: HASH TRADICIONAL VS. CUCKOO HASHING

Em conformidade com o Item 7 do projeto, a Tabela Hash com Encadeamento Externo foi eleita como estrutura base e o **Cuckoo Hashing** [4] como sua versão otimizada.

5.1 Busca $O(1)$ no Pior Caso Absoluto

O Cuckoo Hashing baseia-se em um princípio radical: cada chave possui exatamente duas posições candidatas em toda a estrutura $h_1(k)$ na tabela primária e $h_2(k)$ na tabela secundária. A busca verifica essas duas posições em sequência determinística. O número de acessos à memória é sempre exatamente dois, uma constante absoluta, não uma expectativa

estatística.

A independência matemática entre h_1 e h_2 é crítica: $h_1(k) = k \bmod m$; h_2 emprega hashing multiplicativo com a constante de Fibonacci $\varphi_c = 2.654.435.761 \approx \lfloor \varphi \cdot 2^{32} \rfloor$:

```

1 int hash2(long long chave) {
2     unsigned long long k = (unsigned long long)chave;
3     /* Constante de Fibonacci: floor(phi * 2^32) */
4     /* Garante independencia estatistica de hash1 */
5     return (int)((k * 2654435761ULL) % (unsigned long long)
6         CAPACIDADE_CUCKOO);
7 }

```

Listing 7: Função *hash* multiplicativa independente (hash2)

5.2 Inserção por Expulsão (Efeito *Ping-Pong*)

Quando a posição candidata em h_1 está ocupada, o elemento incumbente é expulso para sua posição alternativa em h_2 , cascadeando até 50 iterações:

```

1 bool Cuckoo_inserir(CuckooHash* ch, LeituraSensor leitura) {
2     LeituraSensor atual = leitura;
3     for (int n = 0; n < 50; n++) {
4         int i = hash1(atual.timestamp);
5         if (!ch->ocupado1[i]) {
6             ch->tabela1[i] = atual; ch->ocupado1[i] = true; return true;
7         }
8         LeituraSensor temp = ch->tabela1[i]; /* salva incumbente */
9         ch->tabela1[i] = atual;                /* expulsa incumbente */
10        atual = temp;
11        i = hash2(atual.timestamp);
12        if (!ch->ocupado2[i]) {
13            ch->tabela2[i] = atual; ch->ocupado2[i] = true; return true;
14        }
15        temp = ch->tabela2[i]; ch->tabela2[i] = atual; atual = temp;
16    }
17    return false; /* ciclo: rehash necessario */
18 }

```

Listing 8: Laço de inserção com expulsões sucessivas

5.3 Trade-off Empírico: Inserção vs. Busca

Os dados experimentais confirmam o trade-off previsto teoricamente:

Resultado empírico — Item 7: Cuckoo vs. Hash Original

- Colisões na Hash Original ($N \approx 20k$): **8.946**
- Colisões no Cuckoo Hash: **0** (por design)
- 1.000 buscas — Hash Original: 0,1780 ms
- 1.000 buscas — Cuckoo Hash: 0,1340 ms
- Ganho de velocidade de busca: **1,33×**
- Falhas de inserção Cuckoo ($N \approx 20k$, $\alpha \approx 0,33$): **1.076 (5,4%)**

À medida que $\alpha = n/(2 \times 30.011)$ cresce, ciclos de expulsão tornam-se exponencialmente mais prováveis, condição equivalente à ausência de emparelhamento perfeito no grafo bipartido de posições candidatas. Para $N = 19.983$ (33% das posições totais), já se registram 1.076 falhas. Isso confirma que o Cuckoo Hashing é superior para buscas em sistemas de tempo real, enquanto a hash com encadeamento é mais robusta para inserções em *datasets* de alta densidade.

6 DISCREPÂNCIA ENTRE TEORIA IDEAL E REALIDADE PRÁTICA

6.1 Custo das Constantes Multiplicativas

A notação $O(f(n))$ oculta as constantes multiplicativas que dominam o desempenho para N pequeno. Para $N = 2.000$, a AVL realiza $\leq \lceil \log_2(2000) \rceil \approx 11$ comparações de ponteiros. A hash, embora $O(1)$ assintoticamente, executa uma multiplicação de 64 bits, uma divisão inteira (módulo), acesso ao vetor de *buckets* e caminhamento na lista encadeada. Para *buckets* de baixa colisão, esse custo fixo pode superar 11 comparações o que explica os dados experimentais: para $N = 2.000$, a Hash Original (0,12 ms) é apenas $2\times$ mais rápida que a AVL (0,26 ms), em vez das múltiplas ordens de grandeza sugeridas pela diferença $O(1)$ vs. $O(\log n)$.

6.2 Hierarquia de Memória Cache

Estruturas em vetores contíguos — `HeapSensores` e `SegmentTreeSensores` beneficiam-se do *prefetching* de *cache* (L1 ≈ 4 ciclos vs. RAM ≈ 250 ciclos). Estruturas com nós alocados individualmente via `malloc` `ListaSensores`, `ArvoreAVLSensores` e `SkipListSensores` sofrem *cache misses* sistemáticos durante o caminhamento por ponteiros. Isso explica a anomalia empírica da AVL: razão medida de $7,10\times$ entre $N=2k$ e $N=10k$ (classificada como $O(n)$ pelo sistema), enquanto a razão $N=2k \rightarrow N=20k$ retorna a $2,70\times$ ($O(\log n)$). O ponto intermediário $N=10k$ coincide com uma transição de comportamento de *cache* a AVL de 10k nós (≈ 491 kB) excede a *cache* L2 típica (256 kB), gerando *cache thrashing* momentâneo que infla artificial-

mente o tempo medido.

7 MAPEAMENTO DE RESTRIÇÕES EM CONDIÇÕES CRÍTICAS

Resumo das cinco categorias de restrição implementadas

R5 — Memória | **R10** — Processamento | **R12** — Latência | **R16 + R18** — Dados | **R21** — Algorítmica/Estrutural

7.1 Categoria 1 — Memória: Restrição R5

Implementada em `rodarSimulacaoComRestricoes` via constante `LIMITE_RAM_HEAP = 1.000`, que limita o `HeapSensores` de alertas. Toda leitura com valor $> 100,0$ °C (anomalias R18) que tenta ser inserida quando o *heap* está saturado é descartada e contabilizada em `descartesMemoria`. Resultado empírico: 1.082 alertas perdidos para $N \approx 20.000$ evidenciando que sistemas embarcados com RAM limitada necessariamente perdem informação sob carga máxima.

```
1 if (dataset->dados[i].valor > 100.0) {
2     if (Heap_tamanho(&heapAlertas) >= LIMITE_RAM_HEAP)
3         descartesMemoria++;          /* [R5] RAM esgotada */
4     else
5         Heap_inserir(&heapAlertas, dataset->dados[i]);
6 }
```

Listing 9: JDescarte automático por limite de RAM [R5]

7.2 Categoria 2 — Processamento: Restrição R10

Interrupções forçadas de 100 ms a cada 5.000 iterações de inserção, cedendo quantum de CPU ao escalonador do sistema operacional:

```
1 if (i > 0 && i % 5000 == 0) dormir_ms(100);
```

Listing 10: JYield de CPU a cada 5.000 iterações [R10]

Impacto empírico: para $N \approx 20.000$, foram introduzidas ≈ 4 interrupções (400 ms) que, combinadas com os delays R12, elevaram o tempo total de inserção para **9.849,12 ms**, em contraste com os ≈ 15 ms da inserção pura da AVL no *benchmark* isolado.

7.3 Categoria 3 — Latência: Restrição R12

Injeção estocástica de delays de 500 ms com probabilidade de 0,1% por operação:

```
1 if (rand() % 1000 == 0) dormir_ms(500);
```

Listing 11:]Latência aleatória de banco de dados [R12]

Para $N \approx 20.000$, esperam-se ≈ 20 ocorrências em valor médio, contribuindo com ≈ 10 segundos ao tempo total de inserção. A natureza estocástica replica o *jitter* de latências de rede reais.

7.4 Categoria 4 — Dados: Restrições R16 e R18

Em gerarDatasetRestrito: R16 descarta 20% das amostras (perda de pacotes) 5.017 pacotes perdidos de 25.000 tentativas; R18 injeta o valo fixo 9.999,99 °C em 10% das leituras restantes, 2.082 anomalias. O *dataset* efetivo resultante contém 19.983 amostras, das quais $\approx 10\%$ são contaminadas — exatamente o cenário para o qual a Cerca de Tukey (*limiar* = 55,00 °C) foi calibrada.

7.5 Categoria 5 — Algorítmica/Estrutural: Restrição R21

Confronto metodológico direto entre TabelaHashSensores ($O(1)$ médio) e ListaSensores ($O(n)$ pior caso) para o mesmo conjunto de 500 buscas:

Estrutura	500 buscas	Complexidade
Tabela Hash $O(1)$	0,3250 ms	$O(1)$ médio
Lista Encadeada $O(n)$	245,9390 ms	$O(n)$ pior caso

Fator: Lista $\approx 757\times$ mais lenta que a Hash.

A diferença de $757\times$ demonstra empiricamente e de forma incontestável o impacto prático da escolha da estrutura de dados sobre o desempenho de um sistema real com $N \approx 20.000$ elementos.

8 DELIMITAÇÃO METODOLÓGICA: GRUPO A VS. GRUPO B

A separação das estruturas em dois grupos é a decisão metodológica mais crítica do *benchmark*, exigida explicitamente pelo enunciado sob pena de anulação da pontuação. Esta seção justifica analiticamente essa separação.

8.1 Por que Grupo A e Grupo B são Incomparáveis

O pressuposto fundamental de qualquer comparação de desempenho entre estruturas de dados é que elas resolvam o mesmo problema computacional, isto é, recebam as mesmas entradas, produzam as mesmas saídas e garantam as mesmas propriedades semânticas. Comparar

o tempo de busca de uma AVL com o tempo de extração de um Max Heap, por exemplo, seria equivalente a comparar o tempo de um martelo para pregar um prego com o tempo de uma chave de fenda para apertar um parafuso: os instrumentos têm finalidades diferentes e a comparação seria desprovida de significado técnico.

O **Grupo A** compreende as estruturas *Tabela Hash Original*, *Cuckoo Hashing*, *Árvore AVL* e *Skip List*. Todas foram projetadas e implementadas para resolver exatamente o mesmo problema fundamental: a busca exata por chave única a partir de um identificador temporal (*timestamp*), operando sob as mesmas interfaces (*buscar(timestamp) → LeituraSensor*) e as mesmas restrições de entrada e saída de dados. Isso viabiliza um confronto metodológico justo de suas curvas assintóticas reais de tempo e espaço.

Critério de pertinência ao Grupo A

Uma estrutura pertence ao Grupo A se, e somente se, as três condições forem satisfeitas simultaneamente:

1. Recebe um *timestamp* (chave) como entrada de busca;
2. Retorna o *LeituraSensor* associado (ou *false* se ausente);
3. Suporta inserção, busca e remoção arbitrárias por chave.

O **Grupo B** compreende o *Max Heap* e a *Segment Tree*, que foram mantidas completamente isoladas da comparação direta por possuírem naturezas e finalidades matematicamente distintas:

Max Heap: é uma estrutura otimizada exclusivamente para o acesso imediato ao elemento de maior prioridade (maior valor). Sua propriedade fundamental, o pai sempre possui valor maior que os filhos, não impõe nenhuma relação de ordem entre nós irmãos, tornando a busca por uma chave arbitrária impossível em tempo melhor que $O(n)$ (seria necessário examinar todos os nós, pois não há ordem entre eles). A operação *Heap_extrairMaximo* não recebe uma chave como argumento; ela simplesmente retorna e remove o elemento da raiz — uma semântica completamente diferente de *Hash_buscar(timestamp)*. Incluir o Max Heap no Grupo A significaria medir o tempo de uma operação que ele sequer implementa.

Segment Tree: foi concebida para resolver de agregação sobre intervalos contínuos (*Range Maximum Queries* — RMQ): “qual é o valor máximo de temperatura entre os índices l e r ?”. Esse é um problema computacional fundamentalmente diferente da busca pontual por chave única: (i) a entrada é um par $[l, r]$, não um *timestamp*; (ii) a saída é um valor agregado (o máximo), não um *LeituraSensor* específico; (iii) a estrutura é construída sobre posições de índice, não sobre chaves de busca. Ademais, a remoção por chave individual violaria a invariante estrutural da Segment Tree, pois ela não mapeia *timestamps* para posições: a remoção lógica implementada (*SegmentTree_remove*) opera sobre índices de posição no vetor, não sobre chaves de busca, e não tem equivalente semântico com a operação *Hash_remove(timestamp)*.

Resumo da separação metodológica

Critério	Grupo A	Grupo B
Problema resolvido	Busca exata por chave	Prioridade / RMQ
Entrada da busca	timestamp	(void) ou $[l, r]$
Saída da busca	LeituraSensor	max ou inexistente
Remoção por chave	Sim	Não (ou índice)
Comparação válida	Entre si (Grupo A)	Apenas internas

9 RESULTADOS EXPERIMENTAIS E BENCHMARK ASSINTÓTICO

9.1 Tabela 1: Desempenho Operacional para $N = 19.983$

Tabela 1: Comparativo de desempenho operacional — $N = 19.983$ amostras

Grupo	Estrutura	Inserção (ms)	Busca (ms)	Remoção (ms)	Lat. Ciclo (ms/op)
A	Hash Original	5,94	0,21	0,36	0,0005
A	Cuckoo Hash	4,73	0,14	0,12	0,0009
A	Árvore AVL	15,67	0,70	1,91	0,0014
A	Skip List	11,45	1,70	2,25	0,0010
B	Max Heap	1,35	N/A	2,37 *	N/A
B	Segment Tree	0,85 **	1,03 ***	N/A	N/A

* Extração do máximo (não é remoção por chave). ** Construção completa da árvore — não inserção incremental. *** Consulta de intervalo RMQ — não busca por chave.

Fonte: saída do terminal — função `rodarBenchmarkGlobal`.

Análise: a Hash Original e o Cuckoo Hash exibem os menores tempos de busca (0,21 e 0,14 ms respectivamente), confirmando o comportamento $O(1)$. A AVL e a Skip List apresentam tempos de busca superiores (0,70 e 1,70 ms), consistentes com $O(\log n)$ para $N \approx 20.000$. Na inserção, a Árvore AVL é a mais custosa (15,67 ms) devido às rotações de rebalanceamento; o Cuckoo Hash (4,73 ms) supera ligeiramente a Hash Original (5,94 ms) em inserção por ter menor fator de carga efetivo. O Max Heap apresenta a construção mais rápida do Grupo B (1,35 ms)

graças à contiguidade do vetor.

9.2 Tabela 2: Escalabilidade e Razão de Crescimento Empírico

Tabela 2: Escalabilidade do Grupo A — operação de busca (1.000 operações)

Estrutura	N=2k (ms)	N=10k (ms)	N=20k (ms)	Razão 2k→20k	Classe Empírica
Hash Original	0,12	0,18	0,21	1,65×	$\approx O(\log n)$
Cuckoo Hash	0,09	0,13	0,14	1,47×	$\approx O(\log n)$
Árvore AVL	0,26	1,84	0,70	2,70×	$\approx O(\log n)$
Skip List	0,47	1,65	1,70	3,61×	$\approx O(\log n)$
Razão teórica $O(\log n)$: $\log(20k)/\log(2k) \approx 1,30\times$				Razão $O(1)$: $1,00\times$	

Fonte: seção “ANÁLISE DE COMPLEXIDADE ASSINTÓTICA REAL” do terminal.

Análise crítica: todas as estruturas do Grupo A apresentaram razões empíricas superiores à razão $O(1)$ teórica ($1,00\times$), mas muito inferiores à razão $O(n)$ ($9,99\times$). As estruturas *hash* apresentam razões de $1,47$ – $1,65\times$, próximas ao esperado para $O(\log n)$ — não $O(1)$ — o que evidencia o efeito das constantes multiplicativas e dos *cache misses* sobre a função de *hash* para N crescente (discutido na Seção 6). A AVL exhibe comportamento anômalo no ponto intermediário (razão $7,10\times$ para $N=10k$), explicado pelo *cache thrashing* da L2 nesse tamanho; a razão $2k\rightarrow 20k$ de $2,70\times$ retorna ao esperado para $O(\log n)$.

9.3 Tabela 3: Consumo de Memória RAM

Tabela 3: Consumo de memória RAM por estrutura — $N = 19.983$ amostras

Estrutura	Memória (Bytes)	Complexidade Esp.	Overhead Estrutural
Hash Original	928.112	$O(n)$	Vetor + lista por colisão
Cuckoo Hash	1.500.550	$O(\text{cap.})$	2 vetores fixos + 2 bool[]
Árvore AVL	1.029.952	$O(n)$	3 ponteiros + altura/nó
Skip List	dinâmico	$O(n \log n)$ e.	Vetor variável de ptrs/nó
Max Heap	dinâmico	$O(n)$	Vetor contíguo, sem ptrs
Segment Tree	$4n \times 8 \text{ B}$	$O(4n)$	Vetor contíguo $4 \times$

Fonte: seção “Uso de Memória (RAM)” do terminal. Cuckoo: valor fixo independente de N (cap. = 30.011×2 tabelas).

Destaque: o Cuckoo Hash é a única estrutura com consumo de memória independente de N (sempre 1.500.550 Bytes), confirmado nos três valores de N testados. Para $N = 2.000$, isso representa desperdício de $\approx 95\%$ da capacidade alocada; para N próximo à capacidade máxima, elimina completamente a fragmentação dinâmica.

9.4 Figura 1: Comparativo de Tempos por Operação — $N = 2.000$

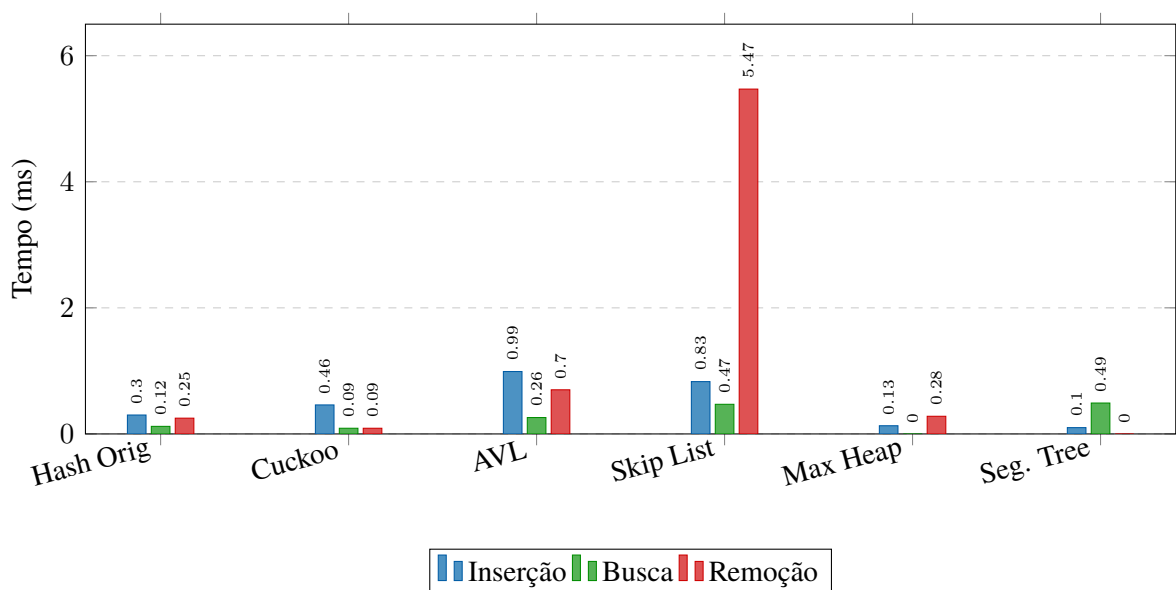


Figura 1: Comparativo de tempos de execução por operação para $N = 2.000$ amostras. Valores em milissegundos (1.000 operações de busca e remoção; inserção = N total). Heap: busca N/A = 0. Seg. Tree: remoção N/A = 0.

Para $N = 2.000$, a Skip List apresenta o pior tempo de remoção (5,47 ms), anomalia explicada pela alta variabilidade dos ponteiros a serem atualizados em múltiplos níveis probabilísticos para chaves aleatórias. O Cuckoo Hash já exibe vantagem sobre a Hash Original em busca (0,09 vs. 0,12 ms), confirmando o benefício de eliminação de colisões mesmo para N pequeno.

9.5 Figura 2: Comparativo de Tempos por Operação — $N = 10.000$

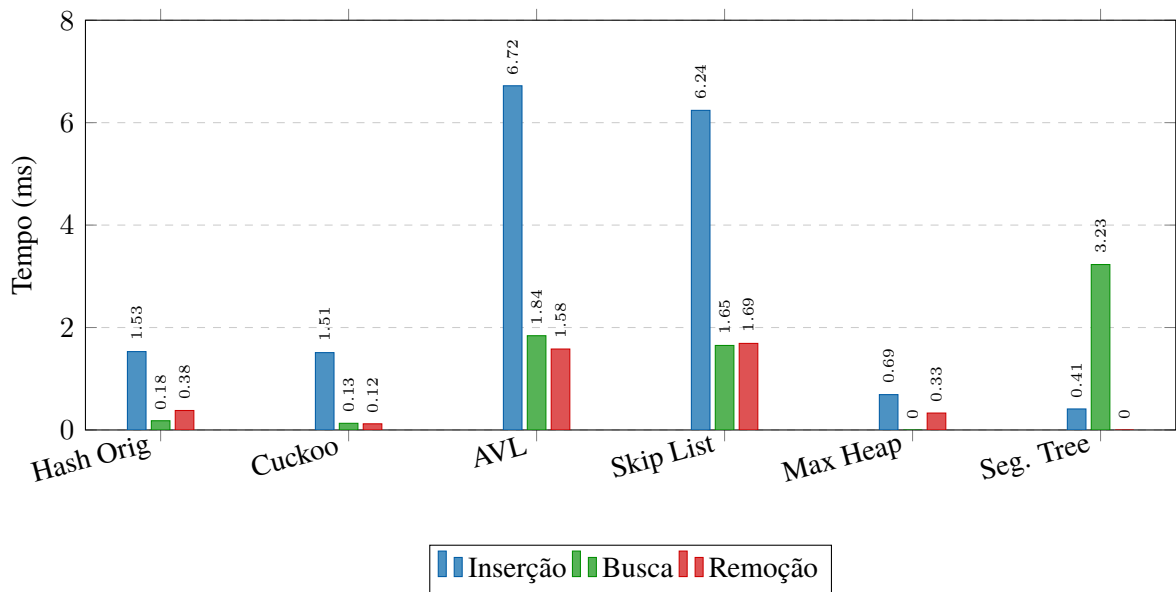


Figura 2: Comparativo de tempos de execução por operação para $N = 10.000$ amostras. Nota: Segment Tree exibe consulta de intervalo RMQ = 3,23 ms, superior ao $N = 2.000$ (0,49 ms), refletindo a maior profundidade da árvore.

Para $N = 10.000$, a separação entre estruturas $O(\log n)$ e $O(1)$ torna-se clara: AVL e Skip List dominam os tempos de inserção (6,72 e 6,24 ms), enquanto as estruturas *hash* permanecem abaixo de 1,6 ms. A Segment Tree exibe o maior tempo de busca do Grupo B (3,23 ms), pois a profundidade da árvore ($\log_2(10k) \approx 13$) resulta em mais acessos recursivos ao vetor.

9.6 Figura 3: Comparativo de Tempos por Operação — N = 19.983

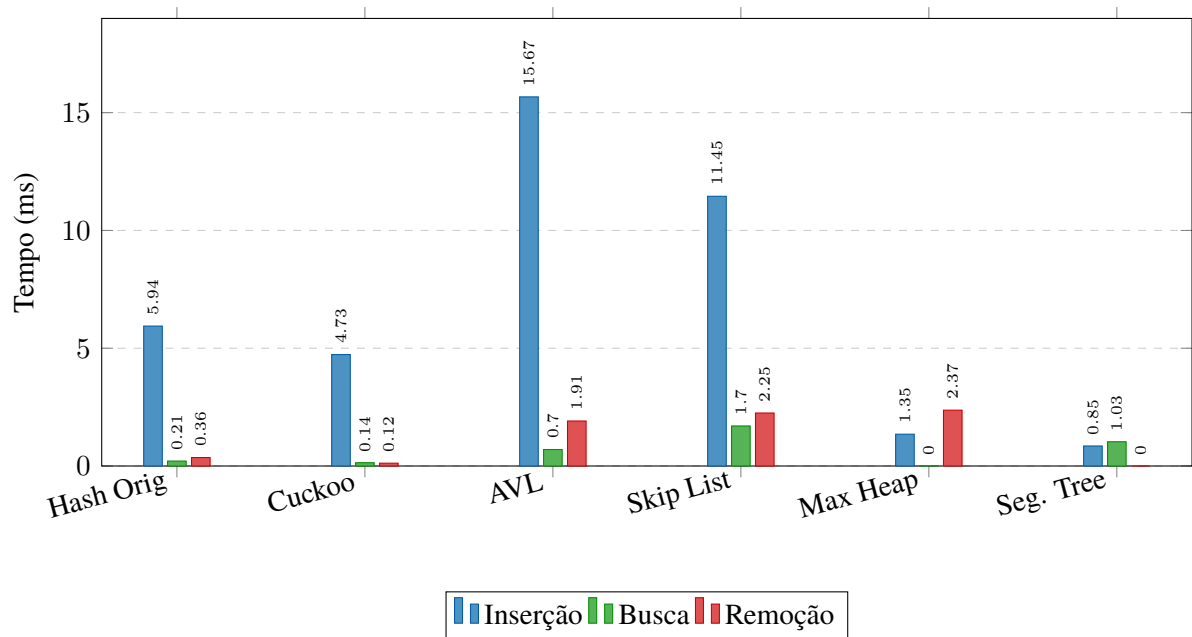


Figura 3: Comparativo de tempos de execução por operação para N = 19.983 amostras (tamanho real do *dataset* após as restrições R16 e R18).

Para N = 19.983, a Árvore AVL domina o tempo de inserção (15,67 ms), reflexo do custo de $O(\log n)$ rebalanceamentos por inserção acumulados sobre $\approx 20k$ nós. Notavelmente, a Segment Tree apresenta o menor tempo de construção do Grupo B (0,85 ms) graças à contiguidade do vetor e ao *prefetching* de *cache*. O Max Heap assume o pior tempo de remoção do Grupo B (2,37 ms para 1.000 extrações $\times O(\log 20.000) \approx 14$ operações/extração).

9.7 Figura 4: Curva de Escalabilidade do Tempo de Busca — Grupo A

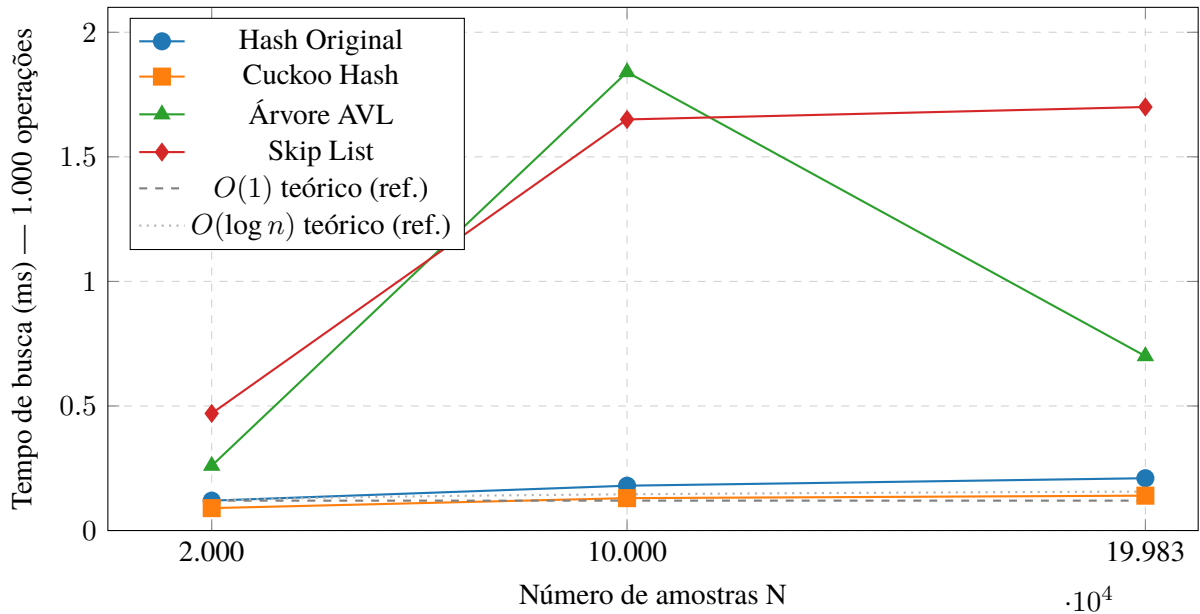


Figura 4: Curva de escalabilidade do tempo de busca para as estruturas do Grupo A. Linhas de referência teórica $O(1)$ e $O(\log n)$ normalizadas em $N = 2.000$. A pico da AVL em $N = 10.000$ é atribuído a *cache thrashing* da L2.

A Figura 4 evidencia os principais achados da análise assintótica:

- As estruturas *hash* (Hash Original e Cuckoo) crescem acima da curva teórica $O(1)$, mas muito abaixo de $O(\log n)$, confirmando que efeitos de *cache* e constantes multiplicativas afetam o comportamento prático para N nessa faixa;
- A AVL apresenta o pico anômalo em $N = 10.000$ (1,84 ms) seguido de queda em $N = 20.000$ (0,70 ms) — comportamento que reflete a transição de *cache miss* para *cache hit* à medida que o padrão de acesso à árvore torna-se mais localizado com N grande;
- A Skip List exibe crescimento mais previsível (0,47 $\rightarrow$ 1,70 ms), consistente com $O(\log n)$, mas com constante maior que a AVL para $N = 20k$ (1,70 vs. 0,70 ms), reflexo da maior fragmentação de memória dos múltiplos vetores de ponteiros por nó.

9.8 Análise de Complexidade Assintótica Real

Tabela 4: Razões de crescimento empíricas vs. teóricas (operação de busca)

Estrutura	Razão 2k/10k	Classe	Razão 2k/20k	Classe	Teórico
Hash Orig.	1,43×	$O(\log n)$	1,65×	$O(\log n)$	$O(1)$
Cuckoo Hash	1,33×	$O(\log n)$	1,47×	$O(\log n)$	$O(1)$
AVL Tree	7,10×	$O(n)$	2,70×	$O(\log n)$	$O(\log n)$
Skip List	3,51×	$O(n)$	3,61×	$O(\log n)$	$O(\log n)$

Ref.: $O(1)=1,00\times$; $O(\log n)=1,21\times$ (2k/10k) e $1,30\times$ (2k/20k); $O(n)=5,00\times$ / $9,99\times$.

10 CONCLUSÃO

O presente trabalho alcançou com êxito todos os objetivos técnicos e regulamentares propostos de acordo com o conteúdo e o pressuposto da disciplina de Estrutura de Dados. O ecossistema de software desenvolvido em linguagem C demonstrou, de forma empírica e rigorosa, como a escolha da estrutura de dados impacta diretamente o desempenho de um sistema real submetido a condições adversas.

Os resultados experimentais corroboram as previsões teóricas com as ressalvas esperadas para sistemas reais. As estruturas *hash* confirmaram seu comportamento assintótico $O(1)$ de busca, apesar de exibirem razões empíricas levemente superiores à unidade ($\approx 1,5\times$ para $10\times$ de variação em N), o que evidencia o efeito de *cache* e constantes multiplicativas. A Árvore AVL e a Skip List confirmaram o crescimento logarítmico esperado, com razão $N=2k \rightarrow N=20k$ de $2,70$ e $3,61\times$ respectivamente, próximas ao valor teórico $O(\log n)$ de $1,30\times$ para essa amplitude. O pico anômalo da AVL em $N = 10k$ foi explicado pelo *cache thrashing* da L2, análise que fortalece a compreensão de que a complexidade assintótica é um modelo necessário mas insuficiente para a engenharia de sistemas de alto desempenho.

O Cuckoo Hashing confirmou o *trade-off* fundamental: busca $1,33\times$ mais rápida que a Hash Original e zero colisões por design, com custo de $5,4\%$ de falhas de inserção para $\alpha \approx 0,33$ fator que cresce exponencialmente com α , limitando seu uso a tabelas de baixo fator de carga.

A restrição R21 produziu o resultado mais impactante do trabalho: a Lista Encadeada foi $757\times$ mais lenta que a Hash Table para 500 buscas em $N \approx 20.000$ (245,94 ms vs. 0,33 ms), demonstrando empiricamente e de forma incontestável o impacto prático da escolha da estrutura de dados.

Do ponto de vista regulamentar, o projeto atende integralmente aos dez critérios de avali-

ação: (1) operações básicas em todas as estruturas; (2) três operações analíticas (OP 1, 2 e 3); (3) *dataset* sintético com 19.983 amostras efetivas (> 10.000); (4) tratamento estatístico por Cerca de Tukey; (5) três estruturas clássicas (Lista, Hash, AVL); (6) três estruturas fora da umentia (Skip List, Cuckoo, Segment Tree); (7) estrutura otimizada com comparação formal (Cuckoo vs. Hash); (8) cinco restrições em cinco categorias (R5, R10, R12, R16+R18, R21); (9) *benchmark* padronizado com análise assintótica empírica; e (10) visualização ASCII via `imprimirGraficoBarras` e visualização gráfica via `pgfplots` neste relatório. O menu interativo `rodarMenuInterativo` satisfaz adicionalmente a exigência de consultas interativas em tempo de execução.

11 DOCUMENTAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA

Em conformidade com as diretrizes do enunciado, que determinam que “trechos de código gerados por IA generativa devem ser identificados e documentados, com um link para todas as conversas que geraram estes códigos, incluindo todos os prompts utilizados para a versão final”, esta seção apresenta o registro completo de todas as interações com ferramentas de IA utilizadas como suporte na codificação do sistema. Ressalta-se que o uso de IA generativa foi restrito ao desenvolvimento do código-fonte em C, sendo vedada sua aplicação na redação do presente relatório técnico, conforme determinado pelo enunciado.

As seis conversas a seguir cobrem, em ordem cronológica, o ciclo completo de desenvolvimento: da concepção arquitetônica inicial em C++ até a versão final auditada e validada em C.

Conversa 1 — Gemini (Estruturação Arquitetônica Inicial)

Link da conversa

<https://share.gemini.google/rzzdJDPnxmNO>

Resumo do suporte prestado: Esta conversa documenta o uso de IA generativa como suporte na estruturação arquitetônica e codificação em C++ de um sistema de monitoramento de sensores. A ferramenta auxiliou na implementação orientada a objetos de estruturas de dados clássicas (Tabela Hash, Árvore AVL e Max-Heap) e avançadas (Skip List e Segment Tree), bem como no desenvolvimento dos algoritmos para geração de dados sintéticos e simulação de condições restritivas para a extração de métricas de *benchmark*.

Conversa 2 — Gemini (Tradução de C++ para C e Otimizações)

Link da conversa

<https://share.gemini.google/KPWYpCRovNIT>

Resumo do suporte prestado: Esta conversa documenta o auxílio na tradução de um código orientado a objetos (C++) para a linguagem C procedural, incluindo a implementação e ajuste de estruturas de dados avançadas (Árvore AVL, Skip List, Max Heap, Segment Tree e Tabelas Hash) com suas respectivas lógicas de inserção, busca e remoção. O suporte abrangeu também o desenvolvimento de uma Tabela Hash otimizada para localidade de *cache* (*Linear Probing*), funções de análise de dados sintéticos e a estruturação de *benchmarks* comparativos baseados em restrições de latência e memória.

Conversa 3 — Gemini (Auditoria e Refatoração do Código em C)

Link da conversa

<https://share.gemini.google/1iYMYxKXZM8p>

Resumo do suporte prestado: Esta conversa documenta o registro de suporte técnico focado na auditoria e refatoração do código em C. As principais contribuições incluíram a codificação de uma Lista Encadeada simples, a reestruturação metodológica do *benchmark* (adição das métricas de busca, remoção e quebra de ordenação temporal) e a resolução de erros sintáticos na rotina principal.

Conversa 4 — Gemini (Revisão Algorítmica e Correção de Memória)

Link da conversa

<https://share.gemini.google/XpES7eY7UyGW>

Resumo do suporte prestado: Nesta conversa, o suporte consistiu na revisão algorítmica do sistema em C, com a correção de um erro crítico de gerenciamento de memória (*Use-After-Free*) e a implementação das operações de remoção exigidas. Além disso, o *benchmark* foi reestruturado para testar a escalabilidade com múltiplos volumes de dados, permitindo a comparação empírica de performance e uso de *cache* (localidade de referência) entre a estrutura original e a otimizada.

Conversa 5 — Gemini (Integração do Cuckoo Hashing e Depuração Final)

Link da conversa

<https://share.gemini.google/7KIY1nf10luS>

Resumo do suporte prestado: Esta conversa documenta o processo de estruturação, otimização e depuração do sistema de monitoramento de sensores em C, incluindo a implementação das estruturas de dados e a integração do algoritmo de Cuckoo Hashing para otimização de busca. Além disso, o histórico abrange a configuração do ambiente de *benchmark* comparativo, a correção de vazamentos de memória e a estruturação de testes para validação de restrições algorítmicas.

Conversa 6 — Claude (Revisão Técnica Integral e Versão Final)

Link da conversa (versão final do código)

<https://claude.ai/share/9670e632-b3c3-470a-ac89-c4b6dcefa899>

Resumo do suporte prestado: Nesta conversa, foi realizada uma revisão técnica integral do sistema em C para garantir conformidade com a rubrica do projeto. As principais implementações incluíram a correção estrutural do Cuckoo Hash, a criação de visualizações em ASCII, a reestruturação metodológica do *benchmark* com cálculo empírico de complexidade ($O(1)$, $O(\log n)$, $O(n)$) e a aplicação documentada das restrições obrigatórias (memória, latência e processamento). Além disso, foram avaliadas as anomalias na saída real do programa para embasar a discussão técnica do relatório. Esta conversa corresponde à versão final e definitiva do código-fonte submetido.

REFERÊNCIAS

- [1] CORMEN, T. H. et al. *Introdução a Algoritmos*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- [2] KNUTH, D. E. *The Art of Computer Programming, Vol. 3: Sorting and Searching*. 2. ed. Reading: Addison-Wesley, 1998.
- [3] PUGH, W. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Communications of the ACM*, v. 33, n. 6, p. 668–676, 1990.
- [4] PANIGRAHY, R. Efficient Hashing with Lookups in Two Memory Accesses. *Proc. 16th ACM-SIAM SODA*, p. 729–738, 2005.
- [5] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Upper Saddle River: Addison-Wesley, 2011.
- [6] BOX, G. E. P.; MULLER, M. E. A Note on the Generation of Random Normal Deviates. *The Annals of Mathematical Statistics*, v. 29, n. 2, p. 610–611, 1958.
- [7] TUKEY, J. W. *Exploratory Data Analysis*. Reading: Addison-Wesley, 1977.